TALENTFORGE CSE POC — PROOF OF CONCEPT

Computer Science & Engineering Domain

10-Week Sprint to Validated Product-Market Fit

EXECUTIVE SUMMARY FOR EXECUTION

POC Objective

Build a **fully operational CSE talent marketplace** that proves: ✅ CSE students can complete coding challenges in a sandbox environment
✅ AI auto-grades code (correctness, complexity, style)
✅ Employers can discover and evaluate verified CSE talent
✅ Psychology matching improves hiring success by 40%+
✅ Unit economics work (7:1 LTV:CAC achieved)

Why CSE First?

- **Largest Market**: 1.2M+ CSE graduates/year in India, BIGGEST pool of engineers
- **Fastest Path to MVP**: Code execution validation is simpler than circuit/CAD validation
- **Proven Demand**: Software hiring never stops, multiple rounds per year per company
- **Easiest to Scale**: Algorithmic problems are standardized, infinitely replicable
- **Best Investor Story**: SaaS software-based assessment is easier to understand than hardware domains
- **Fastest Feedback Loop**: Code runs in <5 seconds vs circuits in 15+ seconds

Success Criteria (Week 10)

- 500 CSE students registered (largest cohort)
- 75 CSE employers onboarded (high hiring velocity)
- 300+ completed coding projects
- ₹40 lakhs MRR (Month 10 run-rate)
- 4.5+ NPS score (students & employers)
- 8 case studies with testimonials
- 12 college partnerships signed

POC Investment: ₹28 lakhs (leanest, fastest to market) **Path to Series A**: Demonstrate 20x growth trajectory by Month 3

PART 1: TECHNICAL ARCHITECTURE FOR CSE

A. Core Tech Stack (CSE-Optimized)

Backend (API Layer)

```
Framework: Node.js + Express (for speed)
Database: PostgreSQL (primary) + Redis (caching)
Real-time: WebSocket for live code execution feedback
File Storage: AWS S3 (code submissions, test artifacts)
Job Queue: Bull/RabbitMQ (async code execution & grading)
Authentication: JWT + OAuth (Google, GitHub)
```

Frontend

```
Web: React.js + Tailwind CSS + Monaco Editor (VS Code-like IDE)
Mobile: React Native (iOS/Android, later phase)
Code Editor: Monaco Editor (Microsoft, production-grade)
Syntax Highlighting: Prism.js or Monaco built-in
```

Code Execution Engine (CRITICAL)

```
Primary: Docker-based sandbox (isolates per student per submission)
Supported Languages: Python, Java, C++, JavaScript, Go, Rust
Runtime Limits: 30-second execution timeout, 256MB memory, 1GB disk
Orchestration: Kubernetes + GKE for scaling

Alternative (Lightweight):
- Judge0 API (free, open-source coding platform backend)
- AWS Lambda for serverless code execution
- Replit API for embedded IDE
```

AI/ML Pipeline

```
Code analysis: AST parsing + custom linters for code quality
Time/space complexity: Algorithmic analysis (Big O notation)
Psychology assessment: Big Five + Technical Aptitude + Work Style
Matching engine: XGBoost (skill-to-project matching)
Plagiarism detection: MOSS (Measure of Software Similarity) integration
```

Blockchain (Minimal MVP)

```
Network: Polygon (low gas, ₹0.01 per transaction)
Smart Contracts: ERC-721 NFT badges for problem completion
Wallet: Torus (Aadhaar-linked, one-click onboarding for India)
Storage: IPFS (solution code, test case results)
```

B. CSE-Specific Infrastructure Components

Component 1: Docker-Based Code Execution Sandbox

Architecture:

python

```python
# talentforge_cse/code_executor.py

import os
import docker
import json
import time
from typing import Dict, Tuple
from dataclasses import dataclass

@dataclass
class ExecutionResult:
    success: bool
    stdout: str
    stderr: str
    execution_time: float
    memory_used_mb: int
    exit_code: int
    timed_out: bool

class CodeExecutor:
    """
    Safely executes student code in isolated Docker containers
    Supports: Python, Java, C++, JavaScript, Go, Rust
    """

    def __init__(self, max_timeout=30, max_memory_mb=256):
        self.client = docker.from_env()
        self.max_timeout = max_timeout
        self.max_memory_mb = max_memory_mb

    def execute_code(self,
                     language: str,
                     code: str,
                     test_input: str = "",
                     submission_id: str = None) -> ExecutionResult:
        """
        Execute student code in isolated sandbox
        Returns: execution result with stdout, stderr, exit code, timing
        """

        # Select correct Docker image based on language
        images = {
            'python': 'python:3.11-slim',
```

```
        'java': 'openjdk:17-slim',
        'cpp': 'gcc:11',
        'javascript': 'node:18-alpine',
        'go': 'golang:1.20',
        'rust': 'rust:1.70'
    }

    image = images.get(language.lower())
    if not image:
        return ExecutionResult(
            success=False,
            stdout="",
            stderr=f"Unsupported language: {language}",
            execution_time=0,
            memory_used_mb=0,
            exit_code=1,
            timed_out=False
        )

    # Prepare code for execution based on language
    if language.lower() == 'python':
        entrypoint_code = code
        run_cmd = ['python', '-c', entrypoint_code]

    elif language.lower() == 'java':
        # Java requires class name matching
        # Extract class name from code or use default
        class_name = self._extract_java_class_name(code)
        run_cmd = self._prepare_java_execution(code, class_name)

    elif language.lower() == 'cpp':
        # C++ requires compilation step
        run_cmd = self._prepare_cpp_execution(code)

    elif language.lower() == 'javascript':
        run_cmd = ['node', '-e', code]

    else:
        run_cmd = ['sh', '-c', code]

    try:
        # Create container with resource limits
        container = self.client.containers.run(
            image,
```

```
            command=run_cmd,
            stdin_data=test_input.encode() if test_input else None,
            stdout=True,
            stderr=True,
            detach=True,  # Run in background
            memory=f"{self.max_memory_mb}m",
            memswap=f"{self.max_memory_mb * 2}m",
            cpus=0.5,  # Half CPU max
            network_disabled=True,  # No network access
            read_only=True,  # Read-only filesystem (except /tmp)
            tmpfs={'/tmp': 'size=50m,mode=1777'}
        )

        start_time = time.time()

        # Wait for container to complete (with timeout)
        try:
            exit_code = container.wait(timeout=self.max_timeout)
        except docker.errors.APIError as e:
            if 'deadline exceeded' in str(e).lower():
                container.kill()
                return ExecutionResult(
                    success=False,
                    stdout="",
                    stderr=f"Execution timeout exceeded ({self.max_timeout}s)",
                    execution_time=self.max_timeout,
                    memory_used_mb=self.max_memory_mb,
                    exit_code=-1,
                    timed_out=True
                )
            raise

        execution_time = time.time() - start_time

        # Get output
        logs = container.logs(stdout=True, stderr=True).decode('utf-8', errors='r

        # Get memory stats
        stats = container.stats(stream=False)
        memory_used_mb = stats['memory_stats']['usage'] / (1024 * 1024)

        # Cleanup
        container.remove()
```

```
            # Split stdout and stderr (approximate)
            stdout, stderr = self._parse_output(logs, language)

            return ExecutionResult(
                success=exit_code == 0,
                stdout=stdout,
                stderr=stderr,
                execution_time=execution_time,
                memory_used_mb=int(memory_used_mb),
                exit_code=exit_code,
                timed_out=False
            )

        except Exception as e:
            return ExecutionResult(
                success=False,
                stdout="",
                stderr=f"Execution error: {str(e)}",
                execution_time=0,
                memory_used_mb=0,
                exit_code=1,
                timed_out=False
            )

    def _extract_java_class_name(self, code: str) -> str:
        """Extract public class name from Java code"""
        import re
        match = re.search(r'public\s+class\s+(\w+)', code)
        return match.group(1) if match else 'Main'

    def _prepare_java_execution(self, code: str, class_name: str) -> list:
        """Prepare Java code for execution"""
        # Write to temp file, compile, run
        return [
            'sh', '-c',
            f'echo "{code}" > {class_name}.java && javac {class_name}.java && java {c
        ]

    def _prepare_cpp_execution(self, code: str) -> list:
        """Prepare C++ code for execution"""
        return [
            'sh', '-c',
            f'echo "{code}" | g++ -x c++ -o /tmp/program - && /tmp/program'
        ]
```

```
    def _parse_output(self, logs: str, language: str) -> Tuple[str, str]:
        """Separate stdout from stderr (heuristic)"""
        # In most cases, all output goes to stdout via Docker logs
        # For this POC, we'll assume all is stdout
        return logs, ""

# ============ USAGE EXAMPLE ============
"""
executor = CodeExecutor(max_timeout=30, max_memory_mb=256)

# Python example
code = '''
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
'''

result = executor.execute_code('python', code)
print(f"Success: {result.success}")
print(f"Output: {result.stdout}")
print(f"Time: {result.execution_time:.2f}s")
print(f"Memory: {result.memory_used_mb}MB")
"""
```

Key Features:

- ✅ Fully isolated Docker container per submission
- ✅ Resource limits (256MB memory, 30-sec timeout, 0.5 CPU)
- ✅ Network disabled (security)
- ✅ Supports 6 languages
- ✅ Returns execution time, memory usage, output, errors
- ✅ Timeout detection

Component 2: Auto-Grading Engine

Test Case Evaluation:

python

```python
# talentforge_cse/auto_grader.py

import json
from typing import Dict, List, Tuple

class AutoGrader:
    """
    Auto-grades code submissions against test cases
    Evaluates: correctness, time complexity, space complexity, code style
    """

    def __init__(self, problem_spec: Dict):
        self.problem_spec = problem_spec
        self.test_cases = problem_spec.get('test_cases', [])
        self.expected_complexity = problem_spec.get('expected_complexity', {})  # O(n

    def grade_submission(self, execution_result: 'ExecutionResult', code: str) -> Dic
        """
        Complete grading: correctness + complexity + style + overall score
        Returns: detailed breakdown 0-100
        """

        grade = {
            'total_score': 0,
            'breakdown': {},
            'passed_test_cases': 0,
            'feedback': []
        }

        # ============ PART 1: CORRECTNESS (60 points) ============
        correctness_score = self._grade_correctness(execution_result)
        grade['breakdown']['correctness'] = correctness_score
        grade['total_score'] += correctness_score * 0.60
        grade['passed_test_cases'] = self._count_passed_tests(execution_result)

        if correctness_score < 60:
            grade['feedback'].append(
                f"❌ Correctness: Only {self.passed_test_cases}/{len(self.test_cases)
                f"Check your logic against failing cases."
            )
        elif correctness_score < 100:
            grade['feedback'].append(
                f"⚠️ Correctness: {self.passed_test_cases}/{len(self.test_cases)} tes
```

```
            f"Edge cases may be failing."
        )
    else:
        grade['feedback'].append("✅ Correctness: All test cases passed!")

    # ============ PART 2: TIME COMPLEXITY (20 points) ============
    if not execution_result.timed_out:
        complexity_score = self._grade_time_complexity(execution_result, code)
        grade['breakdown']['time_complexity'] = complexity_score
        grade['total_score'] += complexity_score * 0.20

        if complexity_score < 50:
            grade['feedback'].append(
                f"🐢 Time Complexity: Your solution is too slow (took {execution_
                f"Target: {self.expected_complexity.get('time', 'O(n)')}"
            )
        elif complexity_score < 100:
            grade['feedback'].append(
                f"⏱️ Time Complexity: Solution runs but could be optimized. "
                f"Current: ~{self._estimate_complexity(execution_result)}, Target
            )
        else:
            grade['feedback'].append("⚡ Time Complexity: Optimal!")
    else:
        grade['breakdown']['time_complexity'] = 0
        grade['feedback'].append(
            f"⏱️ Time Limit Exceeded: Your code took >{execution_result.max_timed
            f"Optimize your algorithm."
        )

    # ============ PART 3: SPACE COMPLEXITY (10 points) ============
    space_score = self._grade_space_complexity(execution_result, code)
    grade['breakdown']['space_complexity'] = space_score
    grade['total_score'] += space_score * 0.10

    if space_score < 50:
        grade['feedback'].append(
            f"💾 Space Complexity: Using {execution_result.memory_used_mb}MB. "
            f"Target: {self.expected_complexity.get('space', 'O(1)')}"
        )

    # ============ PART 4: CODE STYLE (10 points) ============
    style_score = self._grade_code_style(code)
    grade['breakdown']['code_style'] = style_score
```

```
        grade['total_score'] += style_score * 0.10

        if style_score < 70:
            grade['feedback'].append(
                "📝 Code Style: Add comments, use better variable names, follow conve
            )

        # Round to nearest integer
        grade['total_score'] = round(grade['total_score'])

        return grade

    def _grade_correctness(self, execution_result: 'ExecutionResult') -> int:
        """
        Test against all hidden test cases
        Score: (passed / total) * 100
        """
        if execution_result.timed_out or not execution_result.success:
            return 0

        # This would require running all test cases and comparing output
        # For now, assume passed if exit code 0
        passed_count = len(self.test_cases)  # Simplified
        return int((passed_count / len(self.test_cases)) * 100) if self.test_cases el

    def _count_passed_tests(self, execution_result: 'ExecutionResult') -> int:
        """Count how many test cases passed"""
        if execution_result.timed_out or not execution_result.success:
            return 0
        return len(self.test_cases)

    def _estimate_complexity(self, execution_result: 'ExecutionResult') -> str:
        """Estimate Big O from execution time"""
        time = execution_result.execution_time
        if time < 0.01:
            return "O(1)"
        elif time < 0.1:
            return "O(log n)"
        elif time < 1:
            return "O(n)"
        elif time < 10:
            return "O(n log n)"
        else:
            return "O(n²) or worse"
```

```python
    def _grade_time_complexity(self, execution_result: 'ExecutionResult', code: str)
        """
        Grade time complexity based on execution time
        Expected time should be specified in problem_spec
        """
        actual_time = execution_result.execution_time
        expected_time = self.expected_complexity.get('max_time', 5.0)  # seconds

        # Perfect: within 80% of expected
        if actual_time < expected_time * 0.8:
            return 100
        # Good: within expected
        elif actual_time < expected_time:
            return 80
        # Fair: up to 2x expected
        elif actual_time < expected_time * 2:
            return 50
        # Poor: > 2x expected
        else:
            return 20

    def _grade_space_complexity(self, execution_result: 'ExecutionResult', code: str)
        """
        Grade space complexity based on memory usage
        """
        memory_used = execution_result.memory_used_mb
        expected_memory = self.expected_complexity.get('max_memory_mb', 50)

        if memory_used < expected_memory * 0.8:
            return 100
        elif memory_used < expected_memory:
            return 80
        elif memory_used < expected_memory * 2:
            return 50
        else:
            return 20

    def _grade_code_style(self, code: str) -> int:
        """
        Grade code style: comments, naming, formatting
        """
        score = 100
```

```
        # Check for comments
        comment_ratio = code.count('#') / len(code.split('\n')) if code else 0
        if comment_ratio < 0.05:
            score -= 20  # Not enough comments

        # Check for long lines (>120 chars)
        long_lines = sum(1 for line in code.split('\n') if len(line) > 120)
        if long_lines > len(code.split('\n')) * 0.3:
            score -= 10

        # Check for poor variable naming (single letters, except i, j)
        poor_vars = len([v for v in self._extract_variables(code)
                         if len(v) == 1 and v not in 'ijn'])
        if poor_vars > 5:
            score -= 15

        return max(0, score)

    def _extract_variables(self, code: str) -> List[str]:
        """Extract variable names from code"""
        import re
        return re.findall(r'\b([a-zA-Z_]\w*)\b', code)

# ============ USAGE EXAMPLE ============
"""
grader = AutoGrader({
    'test_cases': [
        {'input': '5', 'expected_output': '120'},
        {'input': '0', 'expected_output': '1'},
        {'input': '10', 'expected_output': '3628800'}
    ],
    'expected_complexity': {
        'time': 'O(n)',
        'space': 'O(1)',
        'max_time': 0.5,
        'max_memory_mb': 32
    }
})

code = '''
def factorial(n):
    result = 1
    for i in range(1, n+1):
        result *= i
```

```
    return result
print(factorial(int(input())))
'''

result = executor.execute_code('python', code, test_input='5')
grade = grader.grade_submission(result, code)
print(f"Score: {grade['total_score']}/100")
print(f"Feedback: {grade['feedback']}")
"""
```

Grading Breakdown:

- Correctness (60%): Test case passing
- Time Complexity (20%): Execution speed
- Space Complexity (10%): Memory efficiency
- Code Style (10%): Comments, naming, formatting

Component 3: CSE Problem Library (500+ Problems at Launch)

Problem Categories:

Easy (Tier 1 — 100 problems):

- Basic loops & conditionals
- String manipulation
- Array/List operations
- Simple math problems
- Basic sorting/searching

Medium (Tier 2 — 200 problems):

- Data structures (stacks, queues, trees)
- Graph algorithms (BFS, DFS)
- Dynamic programming (easy cases)
- String algorithms (pattern matching)
- Hashing & set problems

Hard (Tier 3 — 200 problems):

- Advanced DP (knapsack, edit distance)

- Tree/Graph DP hybrids
- Greedy + DP combinations
- System design basics
- Competitive programming

Problem Template (JSON):

```json
```

```
{
  "problem_id": "cse_tier2_045",
  "title": "Longest Substring Without Repeating Characters",
  "difficulty": "Medium",
  "domain": "strings",
  "tier": 2,
  "duration_minutes": 25,
  "xp_reward": 150,

  "problem_statement": "Given a string s, find the length of the longest substring wi

  "examples": [
    {
      "input": "abcabcbb",
      "expected_output": "3",
      "explanation": "The answer is 'abc', with length 3."
    },
    {
      "input": "bbbbb",
      "expected_output": "1",
      "explanation": "The answer is 'b', with length 1."
    }
  ],

  "test_cases": [
    {"input": "abcabcbb", "output": "3"},
    {"input": "bbbbb", "output": "1"},
    {"input": "pwwkew", "output": "3"},
    {"input": "", "output": "0"},
    {"input": "au", "output": "2"},
    {"input": "dvdf", "output": "3"}
  ],

  "constraints": {
    "time_limit": 1.0,
    "memory_limit": 64,
    "input_size_range": "1 to 50,000"
  },

  "expected_complexity": {
    "time": "O(n)",
    "space": "O(min(m, n))",
    "explanation": "m = character set size, n = string length"
```

```
  },

  "hints": [
    "Consider using a sliding window approach",
    "Track character positions with a hash map",
    "Expand window when new character found, contract when duplicate"
  ],

  "topics": ["strings", "sliding_window", "hash_table"],
  "similar_problems": ["cse_tier2_046", "cse_tier2_047"],

  "editorial": {
    "approach": "Two-pointer sliding window with hash map...",
    "time_complexity_proof": "Each character visited at most twice...",
    "code_template": "def lengthOfLongestSubstring(s):\n    # Your code here\n    pas
    "reference_solution": "https://github.com/talentforge/solutions/cse_tier2_045"
  }
}
```

Component 4: Psychology + CSE Matching

CSE-Specific Personality Scoring:

```
python
```

```python
def calculate_cse_project_fit(student_profile: Dict, problem: Dict) -> float:
    """
    Matches student psychology to CSE problem
    Returns compatibility score 0-100
    """

    score = 0
    weights = {
        'logical_reasoning': 0.25,      # Problem-solving ability
        'attention_to_detail': 0.20,    # Bug catching
        'learning_velocity': 0.20,      # New concepts
        'persistence': 0.20,            # Stuck on hard problems
        'communication_style': 0.15     # Can explain solutions
    }

    # 1. Logical Reasoning (DSA heavy problems need this)
    student_logic = student_profile.get('logical_reasoning', 50)
    problem_logic_demand = problem.get('logic_demand', 50)  # 0-100
    logic_fit = 100 - abs(student_logic - problem_logic_demand)
    score += logic_fit * weights['logical_reasoning']

    # 2. Attention to Detail (off-by-one errors, edge cases)
    student_detail = student_profile.get('conscientiousness', 50)
    problem_detail_demand = problem.get('edge_case_complexity', 50)
    detail_fit = 100 - abs(student_detail - problem_detail_demand)
    score += detail_fit * weights['attention_to_detail']

    # 3. Learning Velocity (new concepts vs familiar)
    student_learning = student_profile.get('openness', 50)
    problem_novelty = problem.get('novelty_score', 50)  # 0-100
    learning_fit = 100 - abs(student_learning - problem_novelty)
    score += learning_fit * weights['learning_velocity']

    # 4. Persistence (can they handle frustration?)
    student_persistence = student_profile.get('resilience', 50)
    problem_difficulty = problem.get('difficulty_numeric', 50)  # 1-100
    persistence_fit = 100 - abs(student_persistence - problem_difficulty)
    score += persistence_fit * weights['persistence']

    # 5. Communication (system design problems need this)
    student_comm = student_profile.get('extraversion', 50)
    problem_comm_need = problem.get('explanation_required', 50)  # 0-100
    comm_fit = 100 - abs(student_comm - problem_comm_need)
```

```
    score += comm_fit * weights['communication_style']

    return round(score, 1)

# Example output:
# Student: Logical (78), Detail-oriented (82), Learning-hungry (75)
# Problem: "Median of Two Sorted Arrays" (logic 80, detail 75, novelty 70)
# Match Score: 81/100 ✅ RECOMMENDED
```

PART 2: 10-WEEK PROJECT TIMELINE (CSE OPTIMIZED)

PHASE STRUCTURE

```
Week 1:     Setup & Architecture (SAME as ECE)
Week 2-3:   Backend Core + Database (SAME)
Week 4-5:   Code Executor & Auto-Grader (CSE-SPECIFIC, FASTER)
Week 6-7:   Frontend + Gamification (SAME)
Week 8:     Beta Testing (75 users — CSE has larger cohort)
Week 9:     Polish & Optimization
Week 10:    Public Launch + Investor Demo
```

WEEK-BY-WEEK BREAKDOWN

WEEK 1: Foundation & Setup ✅ (Same as ECE)

Monday-Friday:

- [ ] AWS account setup (EC2, RDS, S3, Lambda)
- [ ] GitHub repos created (backend, frontend, smart-contracts)
- [ ] Slack + monitoring tools (DataDog, Sentry)
- [ ] Database schema designed (PostgreSQL)
- [ ] Tech stack finalized (Node.js, React, Docker)
- [ ] System architecture diagram completed
- [ ] API specification (OpenAPI/Swagger)

Deliverables:

- AWS console accessible, running
- GitHub repos with CI/CD pipeline

- Database schema in PostgreSQL
- Complete API specification (60+ endpoints for CSE)

WEEK 2-3: Backend Core + Database ✅ (Same as ECE)

Week 2:

- [ ] User authentication (JWT + OAuth Google/GitHub)
- [ ] Psychometric assessment API (30-min adaptive test)
- [ ] Student profile management

Week 3:

- [ ] Problem listing endpoints
- [ ] Submission & storage (S3)
- [ ] Tier unlock logic
- [ ] Assessment scoring algorithm

Success Metrics:

- 30 beta users complete psychometric test
- User can browse problems by tier
- Files uploaded to S3 working

WEEK 4: Code Executor & Docker Setup ⚡ (CSE HERO)

Day 1-3: Docker Sandbox Setup

- [ ] Docker images for Python, Java, C++, JavaScript, Go, Rust
- [ ] Docker Compose for local dev
- [ ] Memory/CPU/timeout limits configured
- [ ] Test execution with 20 sample problems

Code Setup:

```dockerfile
```

```
# Dockerfile for CSE code executor
FROM python:3.11-slim

RUN apt-get update && apt-get install -y \
    openjdk-17-jdk-headless \
    g++ \
    nodejs \
    golang-go \
    rustc

WORKDIR /app

COPY requirements.txt .
RUN pip install -q \
    docker==6.1.0 \
    psycopg2-binary==2.9.7 \
    redis==4.5.5 \
    boto3==1.26.142

COPY . .

CMD ["python", "-u", "code_executor_worker.py"]
```

Day 4-5: Execution Queue & Async Grading

- [ ] Redis job queue (Bull wrapper)
- [ ] Worker processes submissions
- [ ] Timeout handling (30-sec max)
- [ ] Error logging & retry logic

Success Metrics:

- Execute 10+ problems per second
- Python/Java/C++ all working
- Average execution time < 5 seconds
- <1% execution errors

WEEK 5: Auto-Grader & Test Cases ⚡ (CSE HERO)

Day 1-3: Test Case Evaluation

Day 4-5: Complete Grading Pipeline

- [ ] Correctness scoring (60%)
- [ ] Time complexity scoring (20%)
- [ ] Space complexity scoring (10%)
- [ ] Code style scoring (10%)
- [ ] Detailed feedback generation
- [ ] Score breakdown returned to student

Sample Feedback:

```
Submission Score: 78/100

Breakdown:
✅ Correctness: 8/10 test cases passed (80%)
  └ Failed: Edge case with empty input
⚠️ Time Complexity: O(n²) but target is O(n log n)
  └ Your solution: 2.3 seconds
  └ Optimal solution: 0.2 seconds
✅ Space Complexity: O(1) - Perfect!
📝 Code Style: Add more comments, rename 'x' to 'result'

Next Steps: Optimize your sorting approach using merge sort
```

Success Metrics:

- 100+ problems graded accurately
- Auto-grader accuracy > 99%
- Feedback generated < 3 seconds
- Leaderboard updated in real-time

WEEK 6-7: Frontend + Gamification ✅ (Mostly same as ECE)

Week 6: Code Editor UI

- [ ] Monaco Editor integration (VS Code-like)

- [ ] Syntax highlighting for 6 languages
- [ ] Real-time code execution feedback
- [ ] Test case visibility (pass/fail)
- [ ] Submit button with confirmation

Week 7: Gamification

- [ ] XP system (50-200 XP per problem based on difficulty)
- [ ] Tier progression (Explorer → Apprentice → Practitioner → Expert → Master)
- [ ] Leaderboard (anonymized cohort ranking)
- [ ] Streak system (daily problem solving)
- [ ] Achievement badges

CSE-Specific Gamification:

- "Speed Demon" badge: Solve 3 problems in 1 day
- "Optimal Solution" badge: Achieve 100% on time + space complexity
- "Bug Slayer" badge: First submission 100% acceptance
- "Consistency" streak: Solve 30 days in a row

Success Metrics:

- 100+ students on leaderboard
- Gamification driving 3x+ daily engagement
- NPS > 45 among active users

WEEK 8: Beta Launch (75 CSE Users)

Pre-Beta Checklist:

- [ ] Platform load-tested (200 concurrent users)
- [ ] All user flows working (register → problem → solve → grade → NFT)
- [ ] Payment sandbox ready (Razorpay test mode)
- [ ] Customer support (Discord channel, email)
- [ ] FAQ + onboarding video

Beta Launch:

- [ ] Recruit 75 CSE students (from 8 colleges)
- [ ] Recruit 15 CSE employers (startups, tech companies)
- [ ] Daily standup to catch bugs
- [ ] Feedback surveys (NPS, feature requests)

Beta Timeline:

```
Day 1-2: Soft launch (invite 40 students)
Day 3-4: Ramp up (add 35 more students + 15 employers)
Day 5-7: Monitor & iterate
```

Target Metrics:

- 60+ daily active users
- 150+ problems completed
- <2% critical bugs
- NPS > 40
- 5 paid employer commitments

WEEK 9: Polish & Optimization

Day 1-3: Performance Tuning

- [ ] Page load time < 1.5 seconds (4G)
- [ ] Code execution feedback < 3 seconds
- [ ] Database query optimization
- [ ] Frontend bundle size < 250KB (gzipped)

Day 4-5: Bug Fixes

- [ ] Fix top 15 reported bugs
- [ ] Improve error messages
- [ ] Mobile responsiveness tested
- [ ] Dark mode added

Success Metrics:

- PageSpeed Insights > 85
- <100ms P95 API latency
- 0 critical bugs

WEEK 10: Public Launch + Investor Demo

Monday-Tuesday: Public Launch

```
"TalentForge Launches CSE Talent Marketplace
AI Auto-Grades Code, Serves 1.2M Annual CSE Graduates in India"
```

Social Media Campaign:

- Day 1: Founder story (why CSE first?)
- Day 2-3: Student success stories (video)
- Day 4-5: Employer ROI stats
- Day 6-7: Launch announcement

Target: 8,000+ organic signups (Week 10)

Wednesday-Friday: Investor Demo

Demo Flow (15 minutes):

```
00:00 — Intro: "The problem with CS hiring in India"
00:30 — Live demo: Student solves a problem
02:00 — Code execution + auto-grading in real-time
03:00 — Gamification: XP, leaderboard, tier progression
04:00 — Employer side: Shortlist candidates, hire
05:00 — Blockchain: NFT credential minting
06:00 — Traction numbers (Week 10 results)
10:00 — Unit economics & path to ₹100Cr ARR
13:00 — Q&A
```

Investor Deck (10 slides, CSE-Specific):

Slide 1: Problem
"1.2M CS graduates/year in India. Hiring takes 2-3 months.
Resumes are fabricated. Companies waste ₹10L per bad hire."

Slide 2: Solution
"Auto-validate coding skills via execution + auto-grading.
No ambiguity. Real code. Real tests."

Slide 3: Product Demo
(Live demo or 2-min video)

Slide 4: POC Traction (Week 10)
- 500 students
- 75 employers
- ₹40L MRR
- 85% problem completion rate
- 4.5 NPS

Slide 5: Unit Economics
- CAC (employer): ₹8K
- LTV (employer): ₹75L (24-month)
- LTV:CAC = 9.4:1 ✓ (BEST in class)
- Gross Margin = 78%

Slide 6: Market
- TAM: ₹400 Cr (CS only, India)
- SAM: ₹80 Cr (Year 3)

Slide 7: Competitive Advantage
- Auto-grading (IP moat)
- 500+ problem library
- Psychology matching
- Blockchain credentials

Slide 8: Financial Path
- Year 1: ₹60 Cr ARR
- Year 2: ₹420 Cr ARR (7x)
- Year 3: ₹980 Cr ARR

Slide 9: Team
- Founder: IIT-M, ex-Flipkart engineer
- CTO: 12 years distributed systems
- Advisor: NASSCOM

```
Slide 10: Ask
- ₹30 Cr Series A
- Valuation: ₹180 Cr (3x multiple on MRR)
```

PART 3: TEAM STRUCTURE & BUDGET

Core Team (10 weeks)

Role	Count	Monthly Cost	Notes
Full-Stack Engineers	3	₹6,00,000	Backend, Frontend, DevOps
Code Execution Engineer	1	₹2,00,000	Docker, containerization
Product Manager	1	₹1,00,000	PRD, roadmap, feedback
UI/UX Designer	1	₹80,000	Code editor UI is critical
QA/Tester	1	₹60,000	Test case validation
Community Manager	0.5	₹25,000	Discord, support
Total	**8.5**	**₹10,65,000/month**	

10-Week Total: ₹1,06,50,000 (₹1.06 Cr)

Infrastructure Costs (Monthly)

Service	Cost
AWS (EC2, RDS, S3, Lambda)	₹40,000
Docker registry & Kubernetes	₹15,000
Monitoring (DataDog)	₹10,000
Email/SMS (SendGrid, Twilio)	₹5,000
Polygon/Blockchain	₹3,000
Domain & SSL	₹2,000

Service	Cost
Total/Month	**₹75,000**
10-Week Total	**₹2,25,000**

One-Time Costs

Item	Cost
Smart contract audit	₹1,50,000
Legal & compliance	₹80,000
Content creation (500 problems)	₹2,00,000
Marketing & PR launch	₹2,00,000
Total	**₹6,30,000**

TOTAL 10-WEEK CSE POC BUDGET

```
Personnel:           ₹1,06,50,000  (76%)
Infrastructure:      ₹2,25,000     (1.6%)
Setup & Operations:  ₹6,30,000     (4.5%)
Contingency (10%):   ₹11,55,000    (8.2%)

TOTAL:               ₹1,26,60,000  (~₹1.27 Cr)

Simplified: ₹1.25-1.3 Crores for 10-week CSE POC
```

Why CSE is ₹3L cheaper than ECE:

- No SPICE/circuit simulation licensing (saves ₹20K/month)
- Faster code execution means less infra (saves ₹15K/month)
- Simpler validation engine (saves 1 FTE = ₹2L)

PART 4: SUCCESS METRICS & KPIs

Week 10 Targets (CSE POC)

Metric	Target	vs ECE
Students	500+	+200 (ECE: 300)
Employers	75+	+25 (ECE: 50)
Problems Completed	300+	+100 (ECE: 200 projects)
MRR	₹40L	+₹15L (ECE: ₹25L)
NPS	4.5+	+0.2 (ECE: 4.3)
Completion Rate	80%+	+5% (ECE: 75%)
Platform Uptime	99.5%+	SAME
Code Execution Latency	<5 sec	Faster than ECE

Why CSE outperforms ECE:

- Larger student pool (1.2M vs 800K)
- Faster feedback (code: 5 sec vs circuits: 15 sec)
- Higher job velocity (CSE hiring never stops)
- Easier psychology matching (more standardized roles)

PART 5: RISK MITIGATION

Risk	Mitigation
Code sandbox escape	Use rootless Docker, seccomp profiles, read-only FS
Slow code execution	Pre-compiled base images, connection pooling, caching
Plagiarism (copy-paste)	MOSS integration + session recording
Student churn	Gamification, daily streaks, visible progress
Employer hesitance	First 5 projects free trial, detailed scorecards

Risk	Mitigation
Scaling to 1M students	Kubernetes auto-scaling, serverless code execution

---

PART 6: GO-LIVE CHECKLIST

2 Weeks Before Launch:

- [ ] 500 problems loaded & tested
- [ ] 75 users in beta with <2% bugs
- [ ] Legal: ToS, Privacy Policy finalized
- [ ] DPDP Act compliance reviewed
- [ ] AWS cost alerts configured

1 Week Before Launch:

- [ ] Infrastructure load-tested (500 concurrent users)
- [ ] Database backed up
- [ ] Support process defined (Discord, email, phone)
- [ ] Press release ready

Launch Day:

- [ ] Website live
- [ ] Email notifications sent
- [ ] Social media posts scheduled
- [ ] Team on standby

FINAL: EXECUTION SUMMARY

10-Week Roadmap

```
WEEK 1:    Setup, Architecture, Team
WEEK 2-3:  Backend Core (Auth, Users)
WEEK 4-5:  Code Executor + Auto-Grader (CSE HERO)
WEEK 6-7:  Frontend Dashboard + Gamification
WEEK 8:    Beta Testing (75 users)
WEEK 9:    Polish & Optimization
WEEK 10:   Public Launch + Investor Demo
```

```
INVESTMENT:  ₹1.25-1.3 Crores (CSE saves ₹7-8L vs ECE)
TEAM:        8-10 engineers + support
SUCCESS MET: 500 students, 75 employers, ₹40L MRR, 4.5 NPS
```

Post-POC Path to Series A

Month 3-4:

- Scale to 15K students, 300 employers
- ARR: ₹60+ Crores (annualized)
- Secure 10-15 corporate pilot contracts
- File patent for auto-grading system

Month 5-6:

- Launch ECE domain (parallel)
- Launch Mechanical domain
- NASSCOM partnership signed

Month 12:

- Series A ready: ₹35-50 Cr round
- ARR: ₹200+ Crores
- Team: 40+ people
- 3 domains live, 50K+ MAU

This CSE POC validates the core hypothesis faster and cheaper than ECE. Series A scales to all engineering domains + non-tech skills.